Universidade do Minho
Aplicações e Serviços de Computação em Nuvem

Trabalho Prático

Relatório Inicial — Checkpoint #1

Aplicação Warracker

Grupo 48

Duarte Pinto Bravo - pg63951
Ivo Costa Sousa - pg63976
João Afonso Almeida Sousa - pg62405
Ricardo Eusebio Cerqueira - pg64022
Tiago Gonçalves Melo - pg65445

Novembro de 2026

Índice

1 Introdução **1**

2 Arquitetura e componentes **1**

3 Funcionalidades e APIs **1**

4 Gargalos de desempenho e pontos únicos de falha **2**

4.1 Potenciais gargalos 2

4.2 Pontos únicos de falha 2

5 Plano de trabalho **2**

6 Questões em aberto e dificuldades de instalação **3**

1 Introdução

O Warracker é uma aplicação web *open-source* para gerir garantias de produtos: guarda as datas de compra e de fim de garantia, os comprovativos (faturas, recibos, manuais) e avisa o utilizador quando uma garantia está prestes a expirar. Este relatório descreve a arquitetura e as APIs da aplicação, discute os seus potenciais gargalos e pontos únicos de falha, e apresenta o plano do grupo para a instalação automática no Google Kubernetes Engine (GKE), incluindo as dúvidas e dificuldades encontradas numa primeira instalação local com contentores.

2 Arquitetura e componentes

A distribuição oficial usa *Docker Compose* com dois serviços (Figura 1):

- `warracker` (imagem `ghcr.io/sassanix/warracker/main`): um único contentor que agrupa vários processos geridos pelo *supervisord*:
  - **Nginx** (porta 80): serve o *frontend* estático (HTML/CSS/JavaScript) e funciona como *reverse proxy*, reencaminhando os pedidos `/api/...` para o Gunicorn;
  - **Gunicorn + Flask** (porta 5000, interna): o *backend* em Python, que implementa a lógica de negócio e a API REST, organizado em *blueprints* (autenticação, garantias, *tags*, administração, documentos, OIDC). Corre como utilizador não privilegiado (`warracker`);
  - **cron**: tarefas periódicas que verificam as garantias a expirar e enviam as notificações (email ou Apprise).

  O supervisord reinicia automaticamente qualquer um destes processos se terminar (`autorestart=true`). No arranque, o contentor aplica as migrações do esquema da base de dados (ficheiros SQL/Python numerados).
- `warrackerdb` (imagem `postgres:15-alpine`): a base de dados PostgreSQL, com os utilizadores, garantias, *tags* e preferências. Só está acessível na rede interna do Compose (porta 5432 não exposta).

Os dados persistentes ficam em dois volumes: `postgres_data` (dados da BD) e `warracker_uploads` (ficheiros enviados pelos utilizadores). A configuração (password da BD, `SECRET_KEY`, SMTP, OIDC) é passada por variáveis de ambiente.

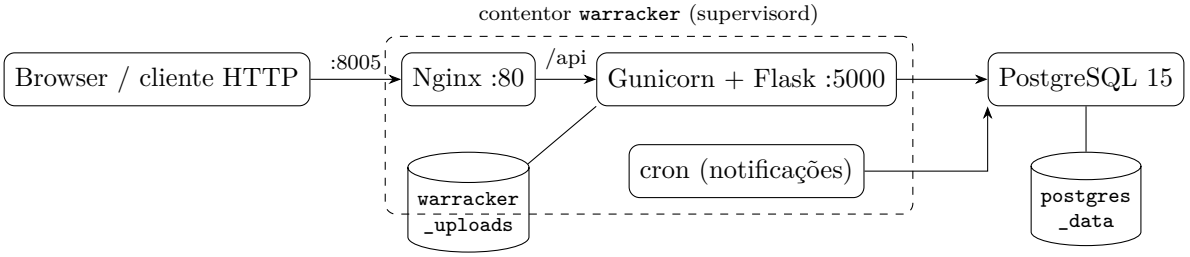


Figura 1: Arquitetura da instalação base do Warracker (Docker Compose).

3 Funcionalidades e APIs

Funcionalidades. Registo de garantias (produto, datas, preço, números de série, notas e garantias vitalícias); envio de comprovativos; *tags*, pesquisa e filtros; registo de reclamações (*claims*); painel de estado com estatísticas; importação e exportação em CSV; notificações de expiração; vários utilizadores, em que o primeiro registado fica administrador; e autenticação local ou por OIDC/SSO; integração opcional com o Paperless-ngx para guardar documentos.

API. O *frontend* comunica com o *backend* apenas por uma API REST com JSON, sob `/api/`, com cerca de 80 rotas distribuídas por 7 *blueprints* Flask (Tabela 1). A autenticação usa JWT: o

login (ou registo) devolve um *token* assinado com a `SECRET_KEY` (HS256), que o cliente envia nos pedidos seguintes no cabeçalho `Authorization: Bearer <token>`.

Grupo (*blueprint*)	Endpoints representativos e função
Saúde	`GET /api/health`: estado do serviço (usado pelo *healthcheck*)
Autenticação	`POST /api/auth/register`, `/login`, `/logout`; perfil, preferências, alteração e recuperação de password
OIDC	`/api/oidc/login`, `/api/oidc/callback`: *login* por SSO
Garantias	`GET|POST /api/warranties`, `PUT|DELETE /api/warranties/<id>`; arquivo, *claims* e importação CSV (`/warranties/import`)
Tags	`GET|POST /api/tags`; associação de *tags* a garantias
Estatísticas	`GET /api/statistics`, `/statistics/global`: painel de estado
Ficheiros	`/api/secure-file/<nome>`: comprovativos; rotas `/api/paperless/...`
Administração	`/api/admin/users`, `/settings`, `/audit-trail`, notificações Apprise

Tabela 1: Organização da API REST do Warracker.

4 Gargalos de desempenho e pontos únicos de falha

4.1 Potenciais gargalos

- **Workers do Gunicorn:** no modo por omissão (*optimized*) há apenas 2 *workers gevent* (confirmado nos *logs*; 4 no modo *performance*, 1 no modo mais leve), escolhidos para poupar memória. Os *workers* gevent tratam vários pedidos em simultâneo enquanto esperam por I/O, mas o processamento em CPU de cada *worker* é sequencial; com muitos clientes, a latência sobe.
- **Pool de ligações à BD:** cada *worker* usa um `SimpleConnectionPool` do `psycopg2` com 1 a 4 ligações (reduzido de 10 para poupar memória). Com 2 *workers*, há no máximo 8 consultas simultâneas à BD, enquanto cada *worker* gevent aceita muitos pedidos ao mesmo tempo. Os pedidos em excesso ficam bloqueados à espera de uma ligação (ou falham), por isso este é, provavelmente, o primeiro limite a ser atingido.
- **Base de dados:** é única e partilhada por todos os pedidos; listagens, pesquisas e estatísticas sobre muitas garantias podem saturar o CPU e o I/O.
- **Uploads e downloads de ficheiros:** passam pelo Flask e pelo disco do volume, consumindo *workers* e I/O durante mais tempo do que um pedido JSON.

4.2 Pontos únicos de falha

- **Instância única do PostgreSQL:** se falhar, toda a aplicação deixa de funcionar.
- **Instância única da aplicação:** Nginx, Flask e agendador estão no mesmo contentor; uma falha nesse contentor (ou no nó onde corre) torna a aplicação inacessível.
- **Volumes:** os dados da BD e os *uploads* estão num único disco; sem cópias de segurança, a perda do disco é a perda dos dados.
- **`SECRET_KEY`:** se não for fixa, é gerada ao acaso a cada arranque e todos os *tokens* deixam de ser válidos; com várias réplicas, todas têm de partilhar a mesma chave.

5 Plano de trabalho

Tarefa 1. Cada componente num *pod* distinto: um *Deployment* para o `warracker` e o PostgreSQL com um *PersistentVolumeClaim* (PVC). Os *uploads* também ficam num PVC. As passwords e a `SECRET_KEY` ficam em *Secrets* e a restante configuração num *ConfigMap*. A aplicação é exposta ao exterior por um *Service* do tipo *LoadBalancer*. Os playbooks Ansible (com a estrutura base da

equipa docente) vão criar o cluster GKE, fazer o *deploy* e permitir parar e retomar a aplicação sem apagar os PVC.

Tarefa 2. Focar a otimização no servidor aplicacional (`warracker`): é o componente que trata todos os pedidos e é *stateless* quanto ao estado de sessão (JWT), por isso é o candidato natural à replicação horizontal (*replicas* e *HorizontalPodAutoscaler*). Também vamos comparar os modos de *workers* do Gunicorn (escalabilidade vertical dentro de cada *pod*). A avaliação vai usar testes de carga que simulam utilizadores típicos (login, listar, criar e pesquisar garantias) com diferentes números de clientes, e as métricas de CPU, RAM e latência do Google Cloud Monitoring.

6 Questões em aberto e dificuldades de instalação

Instalação local. O grupo instalou a aplicação numa VM Ubuntu 24.04 (Vagrant) com o `docker-compose.yml` oficial. O Docker foi instalado com o playbook Ansible do Guião 2; a única dificuldade foi a autenticação SSH entre a VM de controlo e o nó, resolvida autorizando a chave pública da VM de controlo no nó (`ssh-copy-id`). Com a `SECRET_KEY` e a password da BD definidas num `.env`, os dois contentores ficaram *healthy* em cerca de 30 s. O *healthcheck* e o `depends_on` garantem que a aplicação só arranca depois da BD. A API respondeu corretamente (`/api/health`, registo com JWT, conta administradora) e os dados mantiveram-se depois de `docker compose down` e `up`, graças aos volumes. A imagem não inclui ferramentas como `ps`, o que dificulta a inspeção dos processos dentro do contentor.

Questões em aberto para o Kubernetes:

- **Ordem de arranque:** o Kubernetes não tem `depends_on`; será preciso um *initContainer* ou *readiness probes* para a aplicação esperar pela BD.
- **Volume de *uploads* com várias réplicas:** um PVC *ReadWriteOnce* só pode ser montado num nó; replicar o `warracker` exige um volume partilhado (*ReadWriteMany*, p. ex. Filestore) ou outra estratégia de armazenamento.
- **Réplicas e processos internos:** cada réplica aplicaria as migrações no arranque e correria o agendador, com risco de concorrência nas migrações e notificações duplicadas.
- **Imagem com a *tag* `latest`:** convém fixar uma versão para a instalação ser reprodutível.
- **Playbooks base:** a estrutura obrigatória ainda não foi publicada; o plano pode ter de ser ajustado quando o for.